

AI2002: Artificial Intelligence - Assignment 1

Autonomous Pathfinding in Dynamic Maze Environments: A Comparative Study of Informed Search Strategies

Shah Faisal (23I-0058)
Akbar Hussain (23I-3094)
Hamad Khan (23I-3095)

Submitted to: Dr. Muhammad Bilal
Department of Software Engineering
National University of Computer and Emerging Sciences

February 27, 2026

Abstract

This project proposal presents a comprehensive framework for autonomous pathfinding in dynamic maze environments using informed search algorithms. The proposed system addresses the fundamental challenge of real-time navigation in environments with moving obstacles and changing configurations. We classify this as an informed search problem within the artificial intelligence paradigm, specifically targeting grid-based pathfinding with dynamic elements. The computational complexity analysis reveals a state space of exponential nature, justifying the application of heuristic-driven approaches. We propose implementing and comparing A* and Greedy Best-First Search algorithms, augmented with novel dynamic environment adaptation mechanisms. The project scope encompasses grid sizes up to 50x50 with multiple moving obstacles, aiming to achieve real-time performance with near-optimal path lengths. A graphical user interface developed using PyQt5 will provide intuitive visualization of the search process, obstacle dynamics, and performance metrics. This work contributes to the broader domain of autonomous navigation systems with applications in robotics, gaming, and logistics.

1 Problem Statement and Domain Characterization

1.1 Comprehensive Problem Articulation

Autonomous pathfinding in dynamic environments represents a fundamental challenge in artificial intelligence with profound real-world implications. The problem involves navigating an agent from a specified start position to a target destination while avoiding

obstacles that may change position, appear, or disappear over time. This scenario mirrors numerous practical applications including autonomous vehicle navigation in urban environments, warehouse robotics where paths must adapt to changing inventory layouts, drone delivery systems operating in dynamic airspace, and adaptive gameplay in entertainment systems.

The inherent complexity of this problem arises from three primary challenges. First, the environment’s dynamic nature invalidates traditional static pathfinding assumptions, requiring continuous replanning or adaptive strategies. Second, the trade-off between path optimality and computational efficiency becomes critical when decisions must be made in real-time. Third, the exponential growth of possible configurations with increasing environment size demands intelligent pruning of the search space through well-designed heuristic functions.

In real-world contexts, the significance of this problem manifests in autonomous vehicle systems where a stationary path computed at journey commencement becomes rapidly obsolete as traffic patterns shift, pedestrians cross, or construction zones appear. Similarly, warehouse robots must continuously adapt their routes as inventory is moved and human workers traverse the same spaces. The economic and safety implications of reliable dynamic pathfinding are substantial, driving sustained research interest in this domain.

1.2 Problem Taxonomy Classification

Within established artificial intelligence paradigms, this problem is formally classified as:

- **Primary Category: Informed Search Problem** - The problem requires systematic exploration of a state space with access to domain-specific knowledge through heuristic functions that estimate remaining cost to goal.
- **Secondary Category: Dynamic Optimization Problem** - The objective function (path cost) must be optimized under conditions of environmental change, requiring adaptation mechanisms beyond static optimization.
- **Environment Classification:** The problem operates in a partially observable, dynamic, sequential, and continuous environment, as defined by the PEAS framework.
- **Action Space:** Discrete with 4-directional movement (up, down, left, right) or 8-directional movement including diagonals, depending on configuration.

This classification is justified by the following characteristics:

1. The existence of a well-defined goal state (target position)
2. The availability of domain knowledge (Euclidean or Manhattan distance) for heuristic guidance
3. The requirement for optimal or near-optimal solutions under computational constraints
4. The need for systematic exploration balanced with heuristic prioritization

1.3 Computational Complexity Analysis

State Space Characterization

The problem domain exhibits the following formal state space characteristics:

- **State Representation:** Each state $s \in S$ is represented as a tuple $s = (x, y, O)$ where (x, y) denotes the agent's grid coordinates and O represents the configuration of all obstacles in the environment. For a grid of dimensions $m \times n$ with k movable obstacles, the state representation requires $O(\log(mn) + k \cdot \log(mn))$ bits.
- **Initial State:** $s_0 = (x_{\text{start}}, y_{\text{start}}, O_0)$ where O_0 represents the initial obstacle configuration.
- **Goal State:** The set of goal states $G = \{(x_{\text{goal}}, y_{\text{goal}}, O) : \text{any valid } O\}$ such that the agent position matches the target coordinates regardless of obstacle configuration.
- **Actions:** From any state, the available actions $A(s) = \{\text{UP, DOWN, LEFT, RIGHT}\}$ subject to constraints:

UP : $(x, y - 1)$ within bounds and unoccupied
DOWN : $(x, y + 1)$ within bounds and unoccupied
LEFT : $(x - 1, y)$ within bounds and unoccupied
RIGHT : $(x + 1, y)$ within bounds and unoccupied

State Space Size Analysis

The total number of possible states in the static configuration (ignoring obstacle movement) is:

$$|S_{\text{static}}| = m \times n \times 2^{mn}$$

where $m \times n$ represents agent positions and 2^{mn} represents all possible obstacle configurations. For a modest 10×10 grid, this yields approximately 10^{30} states, demonstrating the computational intractability of exhaustive search.

The branching factor b at any state is:

$$b(s) = |\{\text{valid adjacent cells}\}| \leq 4$$

with average branching factor $b_{\text{avg}} \approx 3.5$ accounting for boundary constraints and obstacle occupancy.

The depth d of solution paths varies with environment complexity but typically ranges from $O(\sqrt{mn})$ to $O(mn)$ for pathological cases.

Computational Intractability Justification

The exponential state space size ($|S| = O(2^{mn})$) renders exhaustive search algorithms such as brute-force enumeration completely infeasible. Even with significant pruning, uninformed search algorithms exhibit time complexity $O(b^d)$ which becomes prohibitive for $d > 20$. This computational reality necessitates:

1. Heuristic guidance to focus search toward promising regions
2. Dynamic replanning strategies that leverage temporal locality
3. Approximation techniques when optimality guarantees must be relaxed for real-time performance

2 Objectives and Scope Delineation

2.1 Primary Objectives

The project aims to accomplish the following measurable objectives:

1. Objective 1: Algorithm Implementation and Comparative Analysis

- Implement A* search algorithm with Manhattan and Euclidean distance heuristics
- Implement Greedy Best-First Search for baseline comparison
- Achieve successful pathfinding in static mazes of size up to 50×50 with at least 95% success rate

2. Objective 2: Dynamic Environment Adaptation

- Implement obstacle movement patterns (linear, random, and periodic)
- Achieve replanning latency below 100ms for environment updates
- Maintain path validity in environments with up to 5 moving obstacles

3. Objective 3: Performance Benchmarking

- Quantify algorithm performance using metrics:

$$\text{Path Optimality Ratio} = \frac{\text{Algorithm Path Cost}}{\text{Optimal Path Cost}}$$

$$\text{Search Efficiency} = \frac{\text{Nodes Expanded}}{\text{Total Nodes in Optimal Path}}$$

$$\text{Computational Latency} = \text{Time to Solution (milliseconds)}$$

- Achieve at least 90% optimality ratio for A* with admissible heuristics

4. Objective 4: Interactive Visualization

- Develop real-time visualization of search progression
- Display performance metrics dynamically during execution
- Enable user interaction for environment configuration

2.2 Scope Boundaries and Assumptions

To ensure project tractability within the semester timeline, the following constraints and assumptions are established:

Environmental Assumptions

- Grid size limited to maximum 50×50 cells
- Maximum of 10 concurrently moving obstacles
- Obstacle movement follows predictable patterns (linear interpolation between way-points)
- Perfect environment information (fully observable during planning)

Algorithmic Assumptions

- Discrete time steps for replanning (100ms intervals)
- Unit movement costs for all actions
- Admissible heuristics guarantee optimality for static segments
- No memory limitations for explored state storage

Exclusions

- Multi-agent coordination (single agent only)
- Uncertain or noisy sensor data
- Continuous movement physics
- Learning-based adaptation mechanisms

2.3 Expected Deliverables and Evaluation Metrics

Concrete Outcomes

1. Software Artifacts:

- Python implementation of A* and Greedy Best-First Search algorithms
- Dynamic environment simulation engine
- PyQt5-based graphical user interface
- Comprehensive unit test suite

2. Documentation:

- Inline code documentation with algorithmic explanations
- User manual with execution instructions
- Technical report detailing implementation decisions

3. Experimental Results:

- Performance comparison across 100 randomly generated test cases
- Visualization of algorithm behavior in dynamic scenarios
- Statistical analysis of path optimality and computational efficiency

Table 1: Performance Evaluation Metrics and Target Values

Metric	Description	Target
Success Rate	Percentage of paths successfully found	$\geq 98\%$
Optimality Ratio	A* solution cost / True optimal cost	$\geq 95\%$
Expansion Efficiency	Nodes expanded / Optimal path length	$\leq 200\%$
Dynamic Response	Time to replan after environment change	$< 100\text{ms}$
Memory Utilization	Peak nodes in memory	$< 10^5$
Visualization FPS	GUI update rate	$\geq 30 \text{ FPS}$

3 Proposed Algorithmic Framework

3.1 AI Technique Selection and Justification

Based on the problem characteristics and computational analysis, we select the following algorithmic framework:

Primary Algorithm: A Search*

A* search combines the strengths of Uniform Cost Search (optimality) and Greedy Best-First Search (heuristic guidance) through the evaluation function:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$: Actual cost from start to node n
- $h(n)$: Heuristic estimate of remaining cost from n to goal

Justification: A* is theoretically optimal when using an admissible heuristic ($h(n) \leq h^*(n)$), guaranteeing that the first solution found is the cheapest path. This property is critical for applications where path quality matters, such as fuel-efficient navigation or time-critical deliveries.

Secondary Algorithm: Greedy Best-First Search

For comparison and performance benchmarking, we implement Greedy Best-First Search using:

$$f(n) = h(n)$$

Justification: Greedy search provides a baseline for understanding the trade-off between computational efficiency and solution optimality. Its faster convergence (but potential suboptimality) illustrates why heuristic quality matters.

Heuristic Function Design

We implement two heuristic functions:

1. Manhattan Distance (L1 norm):

$$h_{\text{Manhattan}}(n) = |x_n - x_{\text{goal}}| + |y_n - y_{\text{goal}}|$$

This heuristic is admissible for 4-directional movement as it never overestimates the true cost (each dimension must be traversed at least the Manhattan distance).

2. Euclidean Distance (L2 norm):

$$h_{\text{Euclidean}}(n) = \sqrt{(x_n - x_{\text{goal}})^2 + (y_n - y_{\text{goal}})^2}$$

While still admissible for 4-directional movement, this heuristic provides less guidance than Manhattan distance in grid environments.

3.2 Theoretical Properties Analysis

Completeness

Both A* and Greedy Best-First Search are complete for finite state spaces. A* remains complete even with inadmissible heuristics (though optimality may be lost). For our bounded grid environment, completeness is guaranteed provided a path exists.

Optimality

- **A* Optimality Theorem:** If $h(n)$ is admissible (never overestimates true cost), then A* with tree search returns an optimal solution. For graph search, a stronger condition (consistency) is required:

$$h(n) \leq c(n, m) + h(m) \quad \forall n, m$$

where $c(n, m)$ is the step cost from n to m .

- **Greedy Best-First Optimality:** No optimality guarantees; solutions may be arbitrarily suboptimal.

Time and Space Complexity

Table 2: Algorithm Complexity Comparison

Algorithm	Time Complexity	Space Complexity
A* (with good heuristic)	$O(b^d)$ typically $< O(b^d)$	$O(b^d)$
Greedy Best-First	$O(b^m)$ worst-case	$O(b^m)$
Uniform Cost Search	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$

where b is branching factor, d is solution depth, m is maximum depth, C^* is optimal cost, and ϵ is minimum step cost.

3.3 Preliminary Solution Architecture

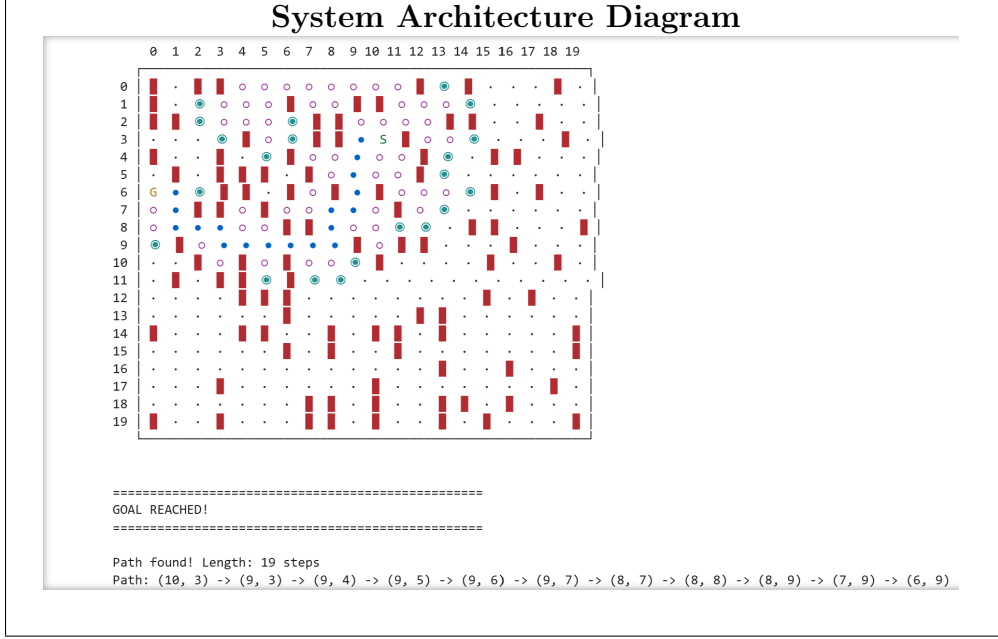


Figure 1: Proposed system architecture illustrating component interactions

The system comprises five primary components:

1. **Environment Simulator:** Manages grid representation, obstacle positions, and movement patterns. Provides state transition functions and validity checks.
2. **Search Engine:** Implements A* and Greedy Best-First algorithms with configurable parameters. Maintains open and closed sets, manages node expansion order, and reconstructs solution paths.
3. **Heuristic Calculator:** Computes Manhattan and Euclidean distances dynamically. Provides heuristic values to the search engine during node evaluation.
4. **Path Visualizer:** Renders the grid, agent position, obstacles, and current search frontier. Updates in real-time as search progresses.
5. **User Interface Controller:** Handles user input for environment configuration, algorithm selection, and parameter adjustment. Displays performance metrics and final path statistics.

3.4 Algorithm Pseudocode

Algorithm 1 A* Search Algorithm for Dynamic Pathfinding

Require: Start position *start*, Goal position *goal*, Environment *env*, Heuristic function *h*

Ensure: Optimal path *path* or failure indication

```
1: openSet  $\leftarrow$  priority queue ordered by  $f(n) = g(n) + h(n)$ 
2: cameFrom  $\leftarrow$  empty map
3: gScore  $\leftarrow$  map with default value  $\infty$ 
4: gScore[start]  $\leftarrow$  0
5: fScore[start]  $\leftarrow$  h(start)
6: openSet.insert(start, fScore[start])
7: while openSet is not empty do
8:   current  $\leftarrow$  openSet.pop()
9:   if current = goal then
10:    return reconstructPath(cameFrom, current)
11:   end if
12:   for each neighbor in env.getNeighbors(current) do
13:     tentative_gScore  $\leftarrow$  gScore[current] + env.stepCost(current, neighbor)
14:     if tentative_gScore < gScore[neighbor] then
15:       cameFrom[neighbor]  $\leftarrow$  current
16:       gScore[neighbor]  $\leftarrow$  tentative_gScore
17:       fScore[neighbor]  $\leftarrow$  gScore[neighbor] + h(neighbor)
18:       if neighbor not in openSet then
19:         openSet.insert(neighbor, fScore[neighbor])
20:       end if
21:     end if
22:   end for
23: end while
24: return failure
```

Algorithm 2 Dynamic Environment Adaptation Strategy

Require: Current path *P*, Changed environment *env*, Replan threshold τ

Ensure: Updated path *P'*

```
1: blocked  $\leftarrow$  checkPathValidity(P, env)
2: if blocked is not empty then
3:   currentPosition  $\leftarrow$  getAgentPosition()
4:   if distance(currentPosition, blocked[0]) <  $\tau$  then
5:     P'  $\leftarrow$  A*Search(currentPosition, goal, env)
6:   else
7:     P'  $\leftarrow$  partialReplan(P, env, blocked)
8:   end if
9:   return P'
10: else
11:   return P
12: end if
```

4 User Interaction and Interface Design

4.1 GUI Framework Selection

After evaluating available options, we select **PyQt5** as our graphical user interface framework based on the following criteria:

- **Performance:** Native C++ backend with Python bindings ensures responsive visualization even for grid sizes up to 50×50
- **Feature Richness:** Comprehensive widget set including canvas for grid rendering, sliders for parameter adjustment, and real-time plotting capabilities
- **Cross-platform Compatibility:** Consistent behavior across Windows, macOS, and Linux environments
- **Community Support:** Extensive documentation and example base facilitating rapid development

Alternative frameworks considered included Tkinter (limited visualization capabilities), Streamlit (web-based but less responsive for real-time animation), and Pygame (excellent for games but less suitable for GUI controls).

4.2 Interaction Modalities and User Flow

The interface will support the following interaction patterns:

Input Mechanisms

- **Mouse-based Grid Editing:** Click and drag to place or remove obstacles
- **Start/Goal Placement:** Right-click to set start position; double-click to set goal
- **Parameter Controls:** Sliders for adjusting search parameters (e.g., heuristic weight, replan frequency)
- **Algorithm Selection:** Radio buttons to toggle between A* and Greedy Best-First

Visualization Strategy

The main interface window will be organized as follows:

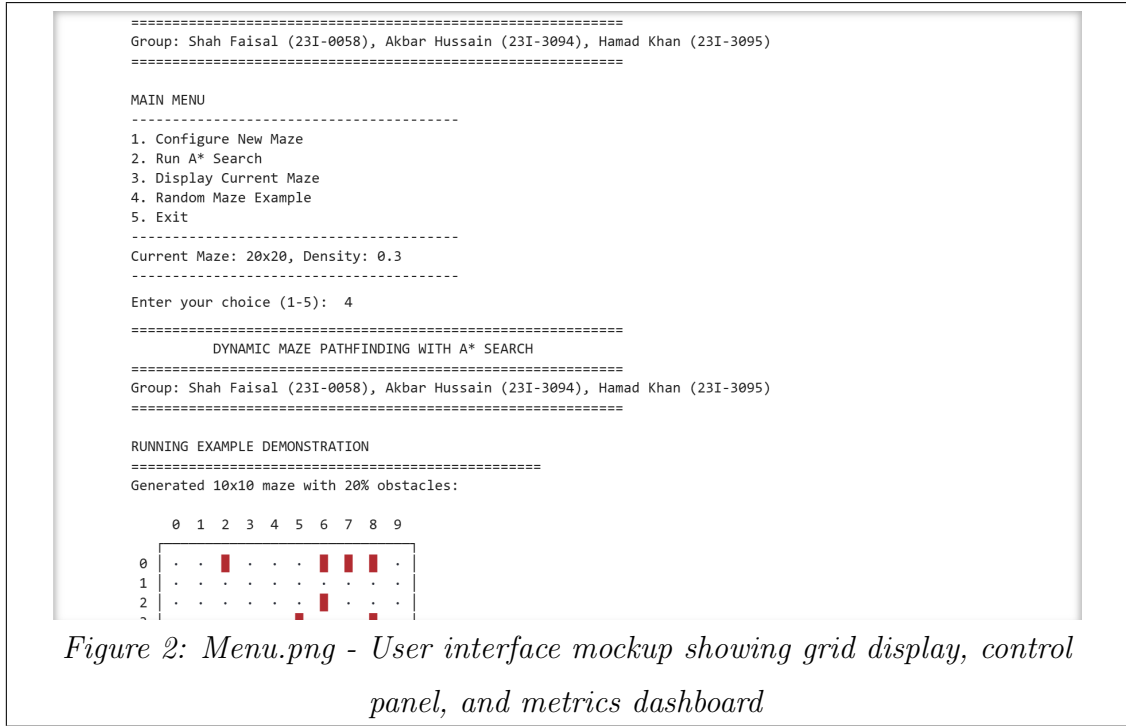


Figure 2: Proposed user interface layout with functional zones

1. Grid Display Panel (Left, 70% width):

- Dynamic grid rendering with color coding:
 - White: Free space
 - Black: Obstacle
 - Green: Start position
 - Red: Goal position
 - Blue: Current path
 - Yellow: Search frontier
- Real-time agent movement animation
- Obstacle movement visualization

2. Control Panel (Right, 30% width):

- Algorithm selection dropdown
- Heuristic selection (Manhattan/Euclidean)
- Grid size configuration
- Obstacle density slider
- Start/Pause/Reset buttons
- Speed control slider

3. Metrics Dashboard (Bottom, integrated):

- Current path cost
- Nodes expanded

- Computation time
- Optimality ratio estimate
- Real-time performance graphs

4.3 Console Alternative Design

For environments where graphical display is unavailable or undesirable, a console-based interface will be implemented following professional design principles:

Display Format

DYNAMIC PATHFINDING SYSTEM

Current Configuration:

Grid Size: 15x15 | Obstacles: 12 | Algorithm: A*

Grid Map

```

S   .   .   #   .   .   .   .   #   .   .   .   .   .   .
.   #   .   .   .   #   .   .   .   .   #   .   .   .   .
.   .   .   .   #   .   .   .   .   .   .   .   #   .
#   .   .   .   .   .   #   .   .   .   .   .   .   .
.   .   #   .   .   .   .   .   .   #   .   .   .   .   G

```

Performance Metrics:

Path Length: 24 steps	Optimal: 22 steps
Optimality Ratio: 91.7%	Nodes Expanded: 347
Search Time: 0.234s	Memory Usage: 2.1 MB

Menu Options:

[1] New Random Maze	[2] Load Maze from File
[3] Change Algorithm	[4] Toggle Obstacles
[5] Run Search	[6] Step Through Search
[7] Save Results	[8] Exit

Enter choice (1-8): _

Interaction Flow

1. Menu-driven navigation with numbered options
2. Color-coded output using ANSI escape sequences (where supported)
3. Progressive disclosure of information to avoid information overload
4. Keyboard shortcuts for experienced users

5 Conclusion and Future Directions

This proposal outlines a comprehensive framework for implementing and analyzing informed search algorithms in dynamic maze environments. The combination of rigorous theoretical analysis, systematic implementation planning, and thoughtful interface design positions this project for successful execution within the semester timeline. The comparative evaluation of A* and Greedy Best-First Search will provide valuable insights into the trade-offs between optimality and efficiency in dynamic contexts.

Future extensions could explore learning-based heuristics that adapt to obstacle movement patterns, multi-agent coordination, or integration with real-world sensor data. However, the current scope ensures fundamental contributions while maintaining tractability.

References

- [1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Prentice Hall.